- Name : Molemi Molokoane
- Email : iimillicent02@gmail.com

Session 9 Assignments: Understanding Model Performance

Section 11: Assignment 1

Q1: A model that classifies transactional fraud has 99.9% accuracy. Why might you still reject this model?

Fraud is extremely rare, so accuracy is dominated by the legitimate transactions. If only 0.1% of transactions are fraud, a "model" that simply predicts *not fraud* for every transaction scores 99.9% accuracy while catching **zero** fraud cases. Accuracy tells us nothing about how the model performs on the class we actually care about. Before accepting it I would look at the confusion matrix and at **recall** (how many real frauds it catches) and **precision** (how many of its fraud flags are real). If recall on the fraud class is low, the model is useless for its purpose no matter how high its accuracy is.

Q2: If a model has a precision of 1.0, does it mean it makes zero errors? Explain.

No. Precision = TP / (TP + FP), so a precision of 1.0 only means there are **no false positives**: every time the model predicts the positive class, it is right. It says nothing about **false negatives**, the positive cases the model missed. A very cautious model that flags only the single most obvious fraud case gets precision 1.0 while missing hundreds of other frauds. To know whether a model makes errors overall you also need recall = TP / (TP + FN) (or the full confusion matrix).

Q3: Explain the difference between MAE and RMSE in your own words.

Both measure how far regression predictions are from the real values, in the same units as the target (e.g. days or Reais).

- **MAE (Mean Absolute Error)** is the plain average size of the errors. Every error counts in proportion to its size: being 10 days off counts exactly ten times as much as being 1 day off. It is easy to explain ("on average we are 2.3 days off") and is not strongly affected by a few outliers.

- **RMSE (Root Mean Squared Error)** squares the errors before averaging and then takes the square root. Squaring makes large errors count much more than small ones, so RMSE punishes a model that is occasionally *very* wrong.

RMSE is always greater than or equal to MAE; the bigger the gap between them, the more the error comes from a few large misses. I would use RMSE when big errors are especially costly and MAE when all errors matter equally or the data has outliers I don't want to dominate the score.

Section 12: Assignment — Choose and Justify the Metric

Scenario 1: Fraud flagging

Task: Binary classification.

Metric: Precision on the fraud class.

Justification: A positive prediction triggers a harsh action: the customer's card is suspended and their order is cancelled immediately. A false positive punishes an honest customer, loses a sale and damages trust in Olist, so we must be confident that whatever we flag really is fraud. I would maximise precision while keeping recall at an acceptable minimum, because a model that never flags anything would also get rid of false positives but would let all fraud through.

Scenario 2: ETA for high-value freight

Task: Regression.

Metric: RMSE.

Justification: A predicted date that is far too early is the expensive mistake here: customers get frustrated and Olist pays severe service penalties. RMSE squares errors, so it penalises large misses much more than MAE and pushes the model away from occasional big errors. Because RMSE treats early and late errors the same way, I would also report how often the prediction is earlier than the real delivery (or use an asymmetric/quantile loss that penalises under-prediction more heavily during training).

Scenario 3: Product category grouping

Task: Multi-class classification.

Metric: Macro-averaged F1-score.

Justification: Putting an item in the wrong category is a moderate inconvenience and no single type of mistake is far more costly than another, so we want a balance of precision and recall, which F1 provides. Olist has some very large categories (e.g. bed_bath_table) and many small ones, so plain accuracy would be dominated by the big categories. Macro averaging computes F1 per category and averages them equally, so the model must also perform well on the small categories.